p0701r1
Back to the std2::future Part II

Published Proposal, 10 September 2017

This version:
http://wg21.link/P0701r1

Authors:
Bryce Adelstein Lelbach (STE||AR Group)
Michał Dominiak (Nokia Networks)
Hartmut Kaiser (STE||AR Group)

Audience:
SG1

Toggle Diffs:
☐ Hide deleted text

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Table of Contents

§ 1. Overview

C++11 introduced asynchronous programming facilities:

- `future`/`shared_future` - Provides access to an asynchronous value.
- `promise` - Produces an asynchronous value.
- `async` - Runs a function asynchronously, producing an asynchronous value.
- `packaged_task` - Packages a function to store its result as an asynchronous value.

The Concurrency TS v1 extended these interfaces, adding:

- `future::then` - Attach continuations to asynchronous values.
- `when_all`/`when_any` - Combine and select asynchronous values.

We think many would agree that futures are the right model for asynchronous handles in C++, and we want composable generic futures. **But**, the futures we have today in both the standard and the Concurrency TS v1 are not as generic, expressive or powerful as they should be.

§ 2. `future`/`promise` Should Not Be Coupled to `std::thread` Execution Agents

Until recently `future` and `thread` were inherently entwined and inseparable. This is due to history: we got `future`s with `.get` first, which - due to how it was specified - required an internal synchronization mechanism to be present inside the future's shared state.

This seemed tolerable, because we had just one type of execution agent (`std::thread`s). In C++17, the parallel algorithms library introduced new kinds of execution agents with weaker forward progress guarantees, although they are not surfaced in the standard library API. We'll add more execution agents with the upcoming Executors TS.

There are many different methods of synchronization. They each have different trade-offs, and users may select a particular mechanism that is a good fit for their needs.

Some synchronization mechanisms will only preserve forward progress guarantees (aka **work properly**) with certain kinds of execution agents. In fact, many types of executors will require the use of a particular set of synchronization mechanisms.

For a parallel tasking system like HPX, using native OS synchronization primitives (`mutex`, `condition_variable`, etc) are problematic, because they block at the OS-thread level, not the HPX-task level, and thus interfere with our userspace scheduling. Thus, we need an `hpx::future`. Other libraries and applications that manage asynchronous operations (see: SYCL, folly, agency) also need their own `future` types for the same reasons.

Likewise, for a networking library, we might need to check for new messages and process outstanding work items while blocking - otherwise, we might never receive and process the message that will change the state of the future we are blocking on:

```
T get()
{
  while (!ready())
  {
    // Poll my endpoint to see if I have any messages;
    // If so, enqueue the tasks described by the messages.
    check_messages();

    // If my task queue is not empty, dequeue some tasks
    // and execute them.
    run_tasks();
  }
  return shared_state->get();
}
```

Such a library would also need its own `future` type.

A future built with the Coroutines TS would also need its own future type (see: cppcoro).

`future`s will obviously interact with executors in a number of ways. This is one of the reasons we have decided to delay integrating the `future` extensions in the Concurrency TS into C++17 or C++Next, because the Executors TS will inform the design of `future`'s continuation mechanism. Originally, the executors group believed we needed a future concept, since executors would create execution agents that would be unable to or would prefer not to use `std::future`, and thus would need their own future type.

We feel that the proliferation of `std::future` implementations in a variety of C++ libraries and the inability to use `std::future` in the Executors TS indicates that `std::future` has failed to become the universal vocabulary type it was intended to be. It’s not truly universal, because the blocking interfaces of `std::future` (`.get` and `.wait`) do not parameterize a synchronization mechanism.

This is a problem, because we **want** `std::future` to be a universal vocabulary type, so that we can easily compose and interoperate with futures from different sources.

We've developed a approach for parameterizing and clarify `std::future` execution semantics (e.g. executors) and synchronization semantics.

The basic premise is to implement `future::get` with `future::then` and to implement `future::then` and `promise::set_value` purely with atomics, which we believe is the universal synchronization language that all execution agents can speak.

For the purposes of this paper, we will use the following `BinarySemaphore` concept to parameterize synchronization semantics:

```
template<typename T>
concept BinarySemaphore = requires(T sem) {
  { sem.wait() } -> void;
  { sem.notify() } -> void;
};
```

where `.wait()` returns strictly after anyone else calls `.notify()`. Only a binary semaphore is necessary for the mechanisms described here.

```
template <typename Semaphore>
    requires BinarySemaphore<Semaphore>
T get(Semaphore sem)
{
  // Avoid creating a semaphore and attaching a continuation if value is
  // already present. grab_value_if_present is exposition only.
  if (auto value = grab_value_if_present())
      return value;

  optional<T> store;

  auto continuation = then(
    [&] (auto value) {
      store = move(value);
      sem.notify();
    },
    inline_executor
    // inline_executor is an executor which invokes work immediately on the
    // calling execution agent.
  );

  sem.wait();

  return move(store.value());
}

T get()
{
  // Uses the semaphore type from the executor associated with this future.
  auto sem = // ...
  return get(sem);
}
```

`.get` default value and type for the semaphore is taken from the executor associated with the `future`/`promise`. We are not necessarily suggesting that executors should define a semaphore type to be used by their execution agents. Instead, the semaphore type could be a property of the execution context, and we could retrieve the execution context through the future. This would avoid adding more functionality to the proposed executor interface.

Additional overloads which explicitly take an executor could also be made available.

The next section describes how we implement `future::then` and `promise::set_value`.

`future::then` and `promise::set_value` with atomics, which allows them to be used with any concurrent or parallel execution agent. Our implementation can be found here. A simplified description follows;

Our shared state consists of a byte of flags, the type-erased continuation and a pointer to the value:

```
struct asynchronous_state
{
    using continuation_storage = // Type-erasure mechanism.
    using executor_storage     = // Type-erasure mechanism.

    enum state_type
    {
        VC = 0b10000, // Value        Changing
        VR = 0b01000, // Value        Ready

        CC = 0b00100, // Continuation Changing
        CR = 0b00010, // Continuation Ready
        CX = 0b00001, // Continuation Executed
    };

    std::atomic<state_type> state;
    continuation_storage    cont;
    executor_storage        exec;
    T*                      value;
};
```

`future::then(Executor exec, Continuation cont)` is implemented as follows:

- Construct the type-erased storage for the continuation and executor.
- Set the "Continuation Changing" bit in the state word via a CAS loop. Since this is a `unique_future`, if anyone else has set this flag, `.then` has been called twice, which is an error, so we throw an exception.
- Swap the type-erased continuation and executor we created earlier with the data members `cont` and `exec`. This operation should only involve a few pointer swaps.
- Set the "Continuation Ready" bit, and the "Continuation Executed" bit if the "Value Ready" bit is set, via a CAS loop.

- If we were the one to set the "Continuation Executed" bit, then execute the continuation stored in `cont` with the value stored in `value` using the executor stored in `exec`.

For the overload of `future::then` which does not take an executor parameter, the executor used is the one that is stored in the shared state when `promise::get_future` is called. This is described in greater detail later in the paper.

`promise::set_value(U&& u)` is implemented as follows:

- Allocate and construct the value.
- Set the "Value Changing" bit in the state word via a CAS loop. If anyone else has set this flag, `.set_value` has been called twice, which is an error, so we throw an exception.
- Swap the pointer to the value we created earlier with the data members `value`.
- Set the "Value Ready" bit, and the "Continuation Executed" bit if the "Continuation Ready" bit is set, via a CAS loop.
- If we were the one to set the "Continuation Executed" bit, then execute the continuation stored in `cont` with the value stored in `value` using the executor stored in `exec`.

§ 3. Where are `.then` Continuations are Invoked?

In our current pre-executor world, it is unspecified where a `.then` continuation will be run. There are a number of possible answers today:

- Consumer Side: The consumer execution agent always executes the continuation. `.then` blocks until the producer execution agent signals readiness.
- Producer Side: The producer execution agent always executes the continuation. `.set_value` blocks until the consumer execution agent signals readiness.
- `inline_executor` Semantics: If the shared state is ready when the continuation is set, the consumer thread executes the continuation. If the shared state is not ready when the continuation is set, the producer thread executes the continuation.
- `thread_executor` Semantics: A new `std::thread` executes the continuation.

The first two answers are undesirable, as they would require blocking, which is not ideal for an asynchronous interface.

This issue is not entirely alleviated by executors. The problem is that it is not clear which execution agent (either the consumer or the producer) passes the `.then` continuation to the executor.

Consider an executor that always enqueues a work item into a task queue associated with the current OS-thread. If the continuation is added to the executor on the consumer thread, the consumer thread will execute it. Otherwise, the producer thread will execute the continuation.

Additionally, this seems counter intuitive:

```
auto i = async(thread_pool, f).then(g).then(h);
```

`f` will be executed on `thread_pool`, but what about `g` and `h`? The could be executed on:

- `inline_executor` Semantics: The current execution agent or the execution agent created by `thread_pool` to execute `f`.
- `thread_executor` Semantics: On new `std::thread`s.

The second option is problematic and probably not what the user intended.

The `thread_executor` answer almost works. It removes ambiguity about where the continuation is run without forcing either the consumer or producer execution agents to block. The only problem is that it forces a particular type of execution agent (`std::thread`) on users.

We propose a similar solution to `thread_executor` approach. The continuation should execute on the executor associated with the `future`/`promise` pair; either the executor passed to `promise::get_future` or the executor of the execution agent calling `promise::get_future` (e.g. the producer execution agent). For a `future` created by `async`, this would be the `executor` passed to `async`. Either the consumer execution agent or the producer execution agent will pass the continuation to the executor (as noted above, this is not deterministic and can be observed).

This **executor propagation** mechanism is intuitive, and gives users flexibility and control:

```
auto i = async(thread_pool, f).then(g).then(h);
// f, g and h are executed on thread_pool.
```

```
auto i = async(thread_pool, f).then(g, gpu).then(h);
// f is executed on thread_pool, g and h are executed on gpu.
```

```
auto i = async(inline_executor, f).then(g).then(h);
// h(g(f())) are invoked in the calling execution agent.
```

To implement this, a type-erased reference to an executor is stored along with the continuation in the shared state (at Toronto 2017, a preference was shown for keeping the executor out of the `future`'s type). Machinery for setting and retrieving the executor of the current execution agent (e.g. a global `get_current_executor`) is also needed - a future paper will describe that machinery in greater detail.

§ 4. Passing `future`s to `.then` Continuations is Unwieldy

The signature for `.then` continuations in the Concurrency TS v1 is:

```
ReturnType(future<T>)
```

The `future` gets passed to the continuation instead of the value so that continuation can handle `future`s that contain exceptions. The `future` passed to the continuation is always ready; `.get` can be used to retrieve the value, and will not block. Unfortunately, this can make `.then` quite unwieldy to work with, especially when you want to use existing functions that cannot be modified as continuations:

```
future<double> f;
future<double> f.then(abs); // ERROR: No std::abs(future<double>) overload.
future<double> f.then([](future<double> v) { return abs(v.get()); }); // OK
```

`future`s would be far more composable if the second line in the above example worked. We should be able to use "future-agnostic" functions as continuations - existing unmodified interfaces, `extern "C"` functions, etc.

`.then` should take continuations that are invocable with `future<T>` and continuations that are invocable with `T`. If the continuation is invocable with both, `future<T>` is passed to the continuation (preferring this over `T` ensures compatibility with user code written using the Concurrency TS v1 `future`).

There are two ways that exceptions could be handled

When `.then` is invoked with a continuation that is only invocable with `T` and the `future` that the continuation is being attached to contains an exception, `.then` does not invoke the continuation and returns a `future` containing the exception. We call this **exception propagation**.

Another `.then` could be added that takes a `Callable` parameter that will be invoked with the `future`'s exception in the case of an error. This paper does not propose such an overload in the interest of simplicity.

§ 5. `when_all` and `when_any` Return Types are Unwieldy

`when_all` has the following signature (Concurrency TS v1, 2.7 [**futures.when_all**] p2):

```
template <typename InputIterator>
future<vector<typename iterator_traits<InputIterator>::value_type>>
when_all(InputIterator first, InputIterator last);

template <typename... Futures>
future<tuple<decay_t<Futures>...>>
when_all(Futures&&... futures);
```

And `when_any` has the following signature (Concurrency TS v1, 2.9 [**futures.when_any**] p2):

```
template <typename Sequence>
struct when_any_result
{
  std::size_t index;
  Sequence futures;
};

template <typename InputIterator>
future<when_any_result<vector<typename iterator_traits<InputIterator>::valu
when_any(InputIterator first, InputIterator last);

template <typename... Futures>
future<when_any_result<tuple<decay_t<Futures>...>>>
when_any(Futures&&... futures);
```

The TL;DR version:

- `when_all` either returns a `future<vector<future<T>>>` or a `future<tuple<future<Ts>...>>`.

- Likewise for `when_any`, with the added complication of the future value type being wrapped in `when_any_result`, which really wants to be a `variant` instead.

Again, the reason for the complexity here is error reporting. If `when_all`'s return type was simplified from `future<vector<future<T>>>` to `future<vector<T>>`, what would we do if some of the `future`s being combined threw exceptions? One possible answer would be for `.get` on the result of `when_all` to throw something like an `exception_list`, where each element of the list would be a `tuple<size_t, exception_ptr>`. An error that occurs during the combination of the `future`s (e.g. in `when_all` itself) could be distinguished by using a distinct exception type.

One benefit of this simplification is that it would enable this pattern:

```
bool f(string, double, int);


future<string> a = /* ... */;
future<double> b = /* ... */;
future<int>    c = /* ... */;

future<bool> d = when_all(a, b, c).then(
  [](future<tuple<int, double, string>> v)
  {
    return apply(f, v); // f(a.get(), b.get(), c.get());
  }
);
```

If `.then` passed a value to the continuation instead of a `future`, as we have proposed, this would become:

```
future<bool> d = when_all(a, b, c).then(
  [](tuple<int, double, string> v)
  {
    return apply(f, v); // f(a.get(), b.get(), c.get());
  }
);
```

We could add a `.then_apply` for `future<tuple<Ts...>>`:

```
future<bool> d = when_all(a, b, c).then_apply(f); // f(a.get(), b.get(), c.
```

`when_any`, clearly, can be updated to use `variant`, which is a natural fit for its interface.

`.then` could be extended to have `visit` like semantics for `when_any` `future`s (e.g. `future<variant<Ts...>>`) in the same way that `.then` could be extended to have `apply` like semantics for `when_all`:

```
future<bool> d = when_any(a, b, c).then_visit(f); // f(a.get()); or f(b.get
```

§ 6. Conditional Blocking in `future`s Destructor Must Go

C++11's `future` will block in its destructor if the shared state was created by `async`, the shared state is not ready and this `future` was holding the last reference to the shared state. This is done to prevent runaway `std::thread`s from outliving main.

These semantics are restricted to `future`s created by `async` because the semantics are not sensible for programmers using `future` and `promise`.

Implicitly blocking, especially in destructors, is very error prone. Even worse, the behavior is conditional, and there is no way to determine if a particular `future`'s destructor is going to block.

In HPX, this is one of the few places where our implementation has chosen to not conform to the standard. We made this decision based on usage experience and feedback from our end-users.

It's time to revisit this design decision. Runaway `std::thread`s should be addressed in another way. `std::future`'s destructor should never block.

§ 7. Immediate Values and `future` Values Should Be Easy to Composable

In C++11, there was no convenience function for creating a future that is ready and contains a particular value (e.g. **immediate values**). You'd have to write:

```
promise<string> p;
future<string> a = p.get_future();
p.set_value("hello");
```

The Concurrency TS v1 adds such a function, `make_ready_future` (Concurrency TS v1, 2.10 [**futures.make_ready_future**]):

```
future<string> a = make_ready_future("hello");
```

However, it is still unnecessarily verbose to work with immediate values. Consider:

```
bool f(string, double, int);


future<string> a = /* ... */;
future<int>    c = /* ... */;

future<bool> d = when_all(a, make_ready_future(3.14), c).then(/* Call f. */
```

Why not allow both `future` and non-`future` arguments to `when_all`? Then we could write:

```
future<bool> d = when_all(a, 3.14, c).then(/* Call f. */);
```

In combination with the direction described in the previous section, we'd be able to write:

```
future<bool> d = when_all(a, 3.14, c).then_apply(f); // f(a.get(), 3.14, c.
```

Additionally, with C++17 class template deduction, instead of `make_ready_future`, we could just have a ready `future` constructor:

```
auto f = future(3.14);
```

§ 8. Proposed Design

```
namespace std2
{

template<typename T>
concept BinarySemaphore = requires(T sem) {
  { sem.wait() } -> void;
  { sem.notify() } -> void;
};

template <typename T>
struct unique_future
{
    using value_type = T;

    constexpr unique_future() noexcept = default;

    unique_future(unique_future&& other) noexcept = default;
    unique_future& operator=(unique_future&& other) noexcept = default;

    unique_future(unique_future const& other) noexcept = delete;
    unique_future& operator=(unique_future const&& other) noexcept = delete

    ////////////////////////////////////////////////////////////////////////
    // Ready Constructor
    unique_future(T const& t);
    unique_future(T&& t);
    // Postconditions: valid() && ready() && get() == t.

    ////////////////////////////////////////////////////////////////////////
    // Continuation Attachment.

    template <typename F>
      requires Callable<F, unique_future<T>> || Callable<F, T>
        auto then(F&& f);
    // Preconditions: ready() == false && valid() == true.
    //
    // Preconditions: then has not been called on *this.
    //
    // Effects: p.execute(std::forward<F>(f)) is invoked on either this
    // execution agent, or on the execution agent that calls set_value on t
```

```
    // unique_promise associated with *this, where p is the executor
    // stored in the shared state when get_future was called on the
    // unique_promise associated with *.this.

    template <typename E, typename F>
      requires Executor<E> && Callable<F>
        auto then(E exec, F&& f);
    // Preconditions: ready() == false && valid() == true.
    //
    // Preconditions: then has not been called on *this.
    //
    // Effects: exec.execute(std::forward<F>(f)) is invoked on either this
    // execution agent, or on the execution agent that calls set_value on t
    // unique_promise associated with *this.

    template <typename F>
      requires Callable<F, unique_future<T>> || Callable<F, T>
        auto then_apply(F&& f)
    {
      return then(
        [f = std::forward<F>(f)] (auto&& v) mutable
        { return std::apply(f, std::forward<decltype(v)>(v)); }
      );
    }
    // Requires: T is a std::tuple.

    template <typename E, typename F>
      requires Executor<E> && Callable<F>
        auto then_apply(E exec, F&& f)
    {
      return then(exec,
        [f = std::forward<F>(f)] (auto&& v) mutable
        { return std::apply(f, std::forward<decltype(v)>(v)); }
      );
    }
    // Requires: T is a std::tuple.

    template <typename F>
      requires Callable<F, unique_future<T>> || Callable<F, T>
        auto then_visit(F&& f)
    {
      return then(
        [f = std::forward<F>(f)] (auto&& v) mutable
```

```
      { return std::visit(f, std::forward<decltype(v)>(v)); }
    );
  }
  // Requires: T is a std::tuple.

  template <typename E, typename F>
    requires Executor<E> && Callable<F>
      auto then_visit(E&& exec, F&& f)
  {
    return then(exec,
      [f = std::forward<F>(f)] (auto&& v) mutable
      { return std::visit(f, std::forward<decltype(v)>(v)); }
    );
  }
  // Requires: T is a std::variant.

  ////////////////////////////////////////////////////////////////////////
  // Value Retrieval.

  auto get();
  template <typename S>
    requires BinarySemaphore<S>
      auto get(S sem);
  // Preconditions: valid() == true.
  //
  // Effects:
  // * wait()s or wait(sem)s until the shared state is ready.
  // * Retrieves the value stored in the shared state.
  // * Releases the shared state.
  //
  // Postconditions: valid() == false && ready() == true.

  ////////////////////////////////////////////////////////////////////////
  // Blocking.

  void wait();
  template <typename S>
    requires BinarySemaphore<S>
      auto wait(S sem);
  // Preconditions: valid() == true.
  //
  // Effects: Waits until the shared state is ready, using either sem or
  // a semaphore whose type is supplied by the executor stored in the sha
```

```
    // state by either unique_promise::get_future or then.
    //
    // Postconditions: ready() == true.

    template <typename R, typename P>
      bool wait_for(chrono::duration<R, P> const& t);
    template <typename S, typename R, typename P>
      requires BinarySemaphore<S>
        bool wait_for(S sem, chrono::duration<R, P> const& t);
    // Preconditions: valid() == true.
    //
    // Effects: Waits until the shared state is ready or until the relative
    // timeout specified by t has expired, using either sem or a a
    // semaphore whose type is supplied by the executor stored in the share
    // state by either unique_promise::get_future or then.
    //
    // Returns: ready().
    //
    // Throws: Timeout-related exceptions.

    template <typename C, typename D>
      bool wait_until(chrono::time_point<C, D> const& t);
    template <typename S, typename C, typename D>
      require BinarySemaphore<S>
        bool wait_until(S sem, std::chrono::time_point<C, D> const& t);
    // Preconditions: valid() == true.
    //
    // Effects: Waits until the shared state is ready or until the absolute
    // timeout specified by t has expired, using either sem or a a
    // semaphore whose type is supplied by the executor stored in the share
    // state by either unique_promise::get_future or then.
    //
    // Returns: ready().
    //
    // Throws: Timeout-related exceptions.

    ////////////////////////////////////////////////////////////////////////
    // Status Observers.

    bool ready() const;
    // Returns: true if the unique_future is ready.

    bool valid() const;
```

```
    // Returns: true if the unique_future is associated with a shared state
};

template <typename T>
struct unique_future_result
{
    using type = T;
};

template <typename T>
struct unique_future_result<unique_future<T>>
{
    using type = typename unique_future<T>::value_type;
};

template <typename T>
using unique_future_result_t = typename unique_future_result<T>::type;

// Deduction guide for implicit unwrapping.
template <typename U>
unique_future(U&& u) -> unique_future<unique_future_result_t<U>>;

template <typename T>
struct unique_promise
{
    constexpr unique_promise() noexcept = default;

    unique_promise(unique_promise&& other) noexcept = default;
    unique_promise& operator=(unique_promise&& other) noexcept = default;

    unique_promise(unique_promise const& other) noexcept = delete;
    unique_promise& operator=(unique_promise const&& other) noexcept = dele

    ////////////////////////////////////////////////////////////////////////
    // Future Retrieval.

    unique_future<T> get_future();
    template <typename E>
      requires Executor<E>
        unique_future<T> get_future(E exec);
    // Effects: If *this has no shared state, creates the shared state.
    // Stores either exec or the executor of the current execution agent in
    // the shared state.
```

```
    //
    // Returns: A unique_future<T> object with the same shared state as
    // *this.
    //
    // Throws: future_already_retrieved if get_future has already been
    // called on a unique_promise with the same shared state as *this.

    void set_value(T const& t);
    void set_value(T&& t);
    void set_exception(exception_ptr e);
    // Effects: Atomically stores either the value t or the exception_ptr e
    // in the shared state and makes that state ready.
    //
    // Throws: future_error if its shared state already has a stored value
    // exception.
    //
    // Note: May invoke p.execute(c), where p is the executor and u is
    // the continuation stored in the shared state associated with *this.
};

}
```